

Capítulo 1

Introducción

Este párrafo comienza después de los tres espacios reglamentarios.

1.1 Introducción

1.2 Antecedentes

1.3 Relevancia de trabajo

1.4 Motivación para la elección del tema

1.5 Formulación del problema

1.6 Objetivo

1.6.1 Objetivo General

1.6.2 Objetivos Específicos

1.7 Metodología

1.8 Orientación al lector sobre el informe

Parte I

MARCO TEÓRICO

Capítulo 2

Fundamentos del Ensayo de impulso (1,2/50 μ s)

El presente capítulo establece el marco normativo y conceptual para el desarrollo del software de adquisición y análisis de impulsos de alta tensión. En primera instancia, se definen los conceptos básicos, y los parámetros de la onda de impulso atmosférico tipo rayo 1,2/50 μ s, tomando como base la norma IEC 60060-1 [X]. Posteriormente, se detallan los procedimientos algorítmicos para el cálculo de dichos parámetros temporales y de amplitud. Finalmente, se exponen los requisitos técnicos exigidos por la norma IEC 61083-2 [X] para la validación de software de evaluación y los principios fundamentales para la estimación de la incertidumbre en sistemas de medición digital.

2.1 Términos para ensayos de impulso de tensión tipo rayo

Antes de definir la onda de impulso tipo rayo, es necesario establecer una serie de términos relacionados con los ensayos de impulso de tensión, que serán necesarios para entender las magnitudes analizadas en el software y en la interpretación de los resultados. Estos términos están definidos en la norma IEC 60060-1 [X] y se exponen a continuación:

Tensión de impulso: Es una tensión transitoria aperiódica aplicada intencionalmente, que usualmente se eleva rápidamente a un valor de cresta y luego cae más lentamente a cero.

Tensión de impulso tipo rayo: Es una tensión de impulso con un tiempo de frente

inferior a 20 μs.

Tensión de impulso tipo rayo completa: Es una tensión de impulso tipo rayo, que no es interrumpida por una descarga disruptiva.

Sobreelevación / oscilación (overshoot): es un incremento de la amplitud de una tensión de impulso debido a una oscilación amortiguada en la cresta, causada por el circuito.

Curva registrada: Es una representación gráfica o digital de los datos de ensayo de una tensión de impulso.

Nivel de base: Es el nivel de un registro de un sistema de medición de impulsos cuando hay una entrada cero al instrumento de registro.

Curva base: Es una estimación de una tensión de impulso tipo rayo completa sin una oscilación superpuesta.

Curva residual $R(t)$: Es la diferencia entre la curva registrada y la curva base.

Valor extremo U_e : Es el valor máximo de la curva registrada medido desde el nivel de base en el mismo sentido que el impulso aplicado.

Máximo de la curva base U_b : Es el valor máximo de la curva base.

Función de tensión de ensayo: Es una función de amplitud-frecuencia que se define para representar la respuesta del aislamiento a impulsos con sobrelevación (overshoot), dada por la Ecuación 2.1, donde f es la frecuencia en MHz.

$$k(f) = \frac{1}{1 + 2,2 \cdot f^2} \quad (2.1)$$

Nota: Aplicar esta función como un filtro paso bajo a la curva de tensión residual permite el cálculo del valor de la tensión de ensayo de la tensión de impulso tipo rayo completa equivalente.

Curva residual filtrada ($R_f(t)$): Es la curva residual filtrada por la función de tensión de ensayo.

Curva de tensión de ensayo $u_t(t)$: Es la suma de la curva base con la curva residual filtrada. Es una representación matemática del proceso de filtrado y no es una entidad física o un impulso equivalente.

Valor de la tensión de ensayo U_t : Es el valor máximo de la curva de tensión de ensayo, medido desde el nivel de base en el mismo sentido que el impulso aplicado.

Magnitud de la sobrelevación β : Es la diferencia entre el valor extremo de la curva registrada (U_e) y el valor máximo de la curva base (U_b).

Magnitud relativa de la sobrelevación β' : Es la relación entre la magnitud de la

sobreelevación y el valor extremo, usualmente expresada como un porcentaje.

Tiempo de frente T_1 : Es un parámetro virtual definido como $1/0,6$ veces el intervalo T_{AB} entre los instantes en que el impulso es 30 % y 90 % del valor de cresta en la curva de tensión de ensayo.

Origen virtual O_1 : Es un instante que precede al punto A de la curva de tensión de ensayo en un tiempo $0,3 * T_1$.

Tiempo hasta el semi-valor T_2 : Es un parámetro virtual definido como el intervalo de tiempo entre el origen virtual, O_1 , y el instante en que la curva de tensión de ensayo ha disminuido a la mitad del valor de cresta de la tensión de ensayo.

Tensión de impulso tipo rayo cortada: Es un impulso tipo rayo que es interrumpido en el frente, cresta o cola por una descarga disruptiva, causando una caída súbita de la tensión.

Instante de corte: Es el instante en el que la extrapolación de la línea entre los puntos del 70 % y 10 % (C y D) en el colapso de la tensión cruza el nivel inmediatamente anterior al colapso.

Tiempo de corte T_C : Es el intervalo de tiempo entre el origen virtual O_1 del pulso y el instante relevante.

2.2 Onda de impulso atmosférico

2.2.1 Onda plena

De acuerdo con la norma IEC 60060-1 [X], la tensión de impulso tipo rayo estándar u «Onda plena» es una tensión de impulso tipo rayo completa y suave, que tiene un tiempo de frente de $1,2 \mu s$, y un tiempo hasta el semi-valor de $50 \mu s$, y se describe como un impulso $1,2/50 \mu s$. Esta onda se caracteriza mediante los siguientes parámetros:

Valor pico (U_t): Es el valor máximo absoluto de tensión alcanzado por el impulso registrado. Si existen oscilaciones superpuestas en la cresta, el valor pico se determina a partir de una «curva base».

Tiempo de frente (T_1): Es un parámetro virtual definido como $1/0,6$ veces el intervalo de tiempo comprendido entre los instantes en los que la tensión alcanza el 30 % y el 90 % del valor pico.

$$T_1 = \frac{T_{AB}}{0,6} = \frac{t_{90} - t_{30}}{0,6} \quad (2.2)$$

Tiempo de cola (T_2): Es un parámetro virtual que representa el intervalo de tiempo transcurrido desde el origen virtual (O_1) hasta el instante en que la curva de tensión

de ensayo ha disminuido al 50 % de su valor de cresta (U_t).

Magnitud relativa de la sobreelevación (β'): La relación entre la magnitud de la sobreelevación y el valor extremo, usualmente expresada como un porcentaje.

Para que un impulso sea considerado como «impulso tipo rayo estándar», comúnmente denominado como impulso 1,2/50 µs, la norma establece que debe tener un tiempo de frente (T_1) de 1,2 µs, y un tiempo hasta el semi-valor (T_2) de 50 µs. Adicionalmente, la norma establece las tolerancias que acepta en los parámetros medidos, indicados en la Tabla 2.1:

Tabla 2.1: Tolerancias de la onda de impulso según IEC 60060-1.

Parámetro	Tolerancia
Tensión de ensayo (U_t)	$\pm 3 \%$
Tiempo de frente (T_1)	$\pm 30 \%$ y para equipos con $U_m > 800 \text{ kV}$: -30% hasta 100%
Tiempo hasta el semi-valor (T_2)	$\pm 20 \%$
Sobreelevación (OS)	$\leq 10 \%$

2.2.2 Onda Cortada

Un impulso tipo rayo estándar cortado u «Onda cortada», se denomina así cuando es interrumpido por un entrehierro externo con un tiempo de corte entre 2 µs y 5 µs. Las características del colapso de tensión durante el corte se determinan por el instante del valor 70 % y 10 % de la tensión de impulso aplicada antes del colapso. La duración del colapso de tensión debe ser mucho más rápida que el tiempo de frente del impulso.

2.3 Ensayo de impulso

El ensayo de impulso es una prueba de alta tensión y corta duración, utilizada para verificar la rigidez dieléctrica y el aislamiento de equipos eléctricos, tales como, transformadores, seccionadores y celdas de media tensión, entre otros.

El impulso tipo rayo se produce usualmente mediante un generador de impulsos Marx, el cual consiste esencialmente en un número de capacitores que se cargan en paralelo desde una fuente de tensión continua de baja tensión, que luego se conmutan en serie, y se descargan a través del objeto de ensayo.

El ensayo de impulso, a nivel general, se realiza aplicando un número determinado de impulsos al objeto bajo ensayo, y registrando la respuesta del mismo mediante

un sistema de adquisición de datos, compuesto por un divisor de tensión (resistivo o capacitivo), un atenuador y un osciloscopio digital.

El análisis de los parámetros del impulso registrado permite evaluar la capacidad del equipo para soportar sobretensiones transitorias, y determinar su capacidad de aislamiento.

La norma IEC 60060-1, en su Anexo B, describe un método para el cálculo de los parámetros de las ondas de impulso tipo rayo estándar (plenas y cortadas) con o sin sobreelevación u oscilaciones superpuestas. Este método se basa en la estimación de una curva base, que representa la forma idealizada de un impulso tipo rayo sin oscilaciones, y el cálculo de una curva residual, que representa las oscilaciones superpuestas. Posteriormente, se aplica un filtro definido por la función de tensión de ensayo (Ecuación 2.1) para obtener la curva de tensión de ensayo, de la cual se calculan los parámetros del impulso.

Mientras que, en el Anexo D de la misma norma, contiene una guía de implementación de software para la analizar la onda impulso. Particularmente, sobre el método de ajuste de la curva base, y la aplicación del filtro digital con la función de tensión de ensayo.

2.4 Estimación de incertidumbre del software

El software de análisis de impulsos es una parte integral del sistema de medición, por lo que es fundamental evaluar su contribución a la incertidumbre total de la medición. La norma IEC 61083-2 [X] establece los requisitos para la validación del software de evaluación de parámetros de impulso, y proporciona un software generador de datos de prueba (TDG, por sus siglas en inglés) que genera formas de onda de prueba, representativas de formas de onda de impulso tanto sintéticas como grabadas, con sus valores de referencia asociados, para verificar el cumplimiento de los requisitos de incertidumbre del software.

El procedimiento de validación consiste en procesar los impulsos de referencia mediante el software bajo prueba, y los parámetros evaluados a partir de los datos procesados deben estar dentro de los límites de aceptación establecidos en el Anexo A de la norma IEC 61083-2. De esta forma, se puede verificar el rendimiento del software.

En el Anexo B de la misma norma, brinda un método para la estimación de la incertidumbre que consiste en seleccionar el conunto de formas de onda relacionadas con el algoritmo del software que se va a cualificar (por ejemplo, impulsos plenos, impulsos cortados, impulsos de corriente, etc), calcular las diferencias entre los valores de referencia x_{REF} y los valores evaluados por el software bajo prueba. La diferencia máxima observada respecto al valor de referencia se utiliza para obtener el primer

componente de la incertidumbre estándar:

$$u_{B71} = \frac{1}{\sqrt{3}} \cdot \max_{i=1}^n \left| \frac{(x_i - x_{REF,i})}{x_{REF,i}} \right| \quad (2.3)$$

o en el caso de β' , cuando la incertidumbre se da en términos absolutos:

$$u_{B71} = \frac{1}{\sqrt{3}} \cdot \max_{i=1}^n |(\beta'_i - \beta'_{REF,i})| \quad (2.4)$$

Por otro lado, la norma indica la incertidumbre expandida (U_x) de cada valor de referencia, en sus tablas Tablas B.1 a B.6. En dichas tablas, la incertidumbre expandida se presenta con un nivel de confianza de aproximadamente el 95 % ($k = 2$). Este componente para la incertidumbre estándar viene dado por:

$$u_{B72} = \frac{1}{2} \cdot \max_{i=1}^n U_{x,i} \quad (2.5)$$

donde $U_{x,i}$ son las incertidumbres expandidas para $x_{REF,i}$.

Finalmente, la incertidumbre estándar del software para el grupo de formas de onda evaluadas, resulta:

$$u_{B7} = \sqrt{u_{B71}^2 + u_{B72}^2} \quad (2.6)$$

Capítulo 3

Instrumentación y adquisición de señales

Este capítulo detalla los fundamentos teóricos de los sistemas de adquisición de datos en el contexto de ensayos de alta tensión. Se aborda la transición del dominio analógico al digital para la captura de ondas de impulso tipo rayo ($1, 2/50 \mu\text{s}$) empleando un osciloscopio de almacenamiento digital.

A lo largo de las siguientes secciones, se analizan los conceptos básicos necesarios para entender el funcionamiento de un osciloscopio.

3.1 Arquitectura de osciloscopios de almacenamiento digital

Antes de procesar digitalmente las señales, es necesario capturarlas adecuadamente. Para ello se utiliza un instrumento de medición electrónico denominado, osciloscopio de almacenamiento digital (DSO, *Digital Storage Oscilloscope*). El cual se encarga de convertir la señal analógica a un formato digital, permitiendo su almacenamiento, visualización y análisis posterior. Por esto es útil comprender su arquitectura, ya que su diseño y limitaciones impactan directamente en la calidad de los datos adquiridos y, por ende, en el análisis posterior.

La señal ingresa por un circuito analógico de acondicionamiento de entrada, para dirigirse al bloque del convertidor Analógico-Digital (ADC, *Analog-Digital Converter*) que se encarga de realizar el muestreo y cuantización a una tasa definida por la base de tiempo. Luego, se almacena en la memoria del osciloscopio, para finalmente pre-

sentarla en pantalla.

A continuación, se describen los parámetros más relevantes relacionados con la adquisición de señales, que deben ser considerados al momento de configurar el osciloscopio para capturar impulsos de alta tensión.

3.1.1 Muestreo de señales

La conversión analógica-digital se basa en el **Teorema de Muestreo** de Nyquist-Shannon, el cual establece que la frecuencia de muestreo (f_s) debe ser mayor al doble de la componente de máxima frecuencia (f_{max}) presente en la señal, para evitar el fenómeno destructivo del aliasing (cuando la velocidad de muestreo es insuficiente, pueden aparecer erróneamente componentes de frecuencia más bajas que la de la señal real, denominadas “Alias”). Matemáticamente, se define como:

$$f_s > 2f_{max} \quad (3.1)$$

3.1.2 Resolución vertical

La resolución del eje vertical de un osciloscopio digital está vinculada con la resolución del ADC que tiene internamente. Mientras que la resolución de un ADC está dada por la cantidad de bits (N) del mismo. Considerando que la tensión de entrada varía entre cero y un valor máximo (V_{max}), matemáticamente, puede expresarse:

$$V_{res} = \frac{V_{max}}{2^N} \quad (3.2)$$

Como el valor máximo de la tensión de entrada puede variar dependiendo de la escala vertical del osciloscopio, conviene expresar la resolución como un porcentaje del valor de plena escala.

$$\text{Resolución \%} = \frac{100}{2^N} \quad (3.3)$$

3.1.3 Cuantización de señales

Mientras que el muestreo discretiza el eje temporal, la **cuantización** divide el eje de amplitudes. Este proceso introduce un error numérico inherente denominado **error de cuantización**, ocasionado por la aproximación del valor real al nivel discreto más cercano. El tamaño del paso de cuantización o resolución del LSB (Bit Menos Significativo, por sus siglas en inglés) está dado por:

$$q = \frac{V_{FSR}}{2^N} \quad (3.4)$$

Donde V_{FSR} representa el rango dinámico de escala completa (FSR) (relación entre el valor máximo y el valor mínimo de la señal que se puede capturar o representar), y N es la resolución en bits del convertidor.

3.1.4 Longitud de memoria

El osciloscopio posee una **longitud de memoria** finita para almacenar datos, que junto con la frecuencia de muestreo, determina la ventana temporal total de captura, es decir, el tiempo que el osciloscopio puede registrar datos a una frecuencia de muestreo dada.

$$T_{acq} = \frac{N_{record}}{f_s} \quad (3.5)$$

Donde:

T_{acq} : tiempo total de adquisición.

N_{record} : longitud de registro.

f_s : frecuencia de muestreo máxima.

Por lo tanto, al momento de configurar el osciloscopio debe asegurarse de que el tiempo de adquisición sea suficiente para capturar el fenómeno completo. Normalmente, estos tres parámetros se ajustan automáticamente mediante la escala de tiempo, por eso es necesario entender que dicha escala debe configurarse acorde a la duración total del fenómeno a capturar. Luego, hay algunos equipos que permiten modificar manualmente la frecuencia de muestreo, para una misma escala de tiempo, pero son menos comunes de encontrar.

3.1.5 Ancho de banda

El **ancho de banda** es un parámetro importante del osciloscopio, que se mide en MHz o GHz, e indica la frecuencia máxima a la que puede responder con precisión. Para garantizar una representación fiel de la señal, el ancho de banda debe ser superior a la frecuencia máxima de la señal que se desea medir.

3.1.6 Velocidad de muestreo

La **velocidad de muestreo** es una medida de la resolución en el tiempo, que se mide en MSa/s o GSa/s, e indica la cantidad de muestras por unidad de tiempo (normalmente en segundos) que captura el instrumento, por lo tanto, define implícitamente la separación horizontal entre dos muestras consecutivas.

3.2 Estándares de comunicación de instrumentos

En el diseño e implementación de sistemas de instrumentación virtual, la transferencia de los datos desde la memoria del osciloscopio hacia la computadora requiere de interfaces físicas estandarizadas, y protocolos lógicos de control.

Actualmente, la comunicación entre instrumentos de medida y sistemas de adquisición se rige por los siguientes estándares físicos:

- **GPIB (General Purpose Interface Bus / IEEE-488):** Es un estándar de bus de datos digital de corto rango diseñado específicamente para interconectar y controlar dispositivos de prueba y medida, tales como osciloscopios, multímetros y generadores de señal, mediante un controlador central.
- **Comunicaciones Serie (RS-232 y USB):** Los datos resultantes del muestreo y la digitalización suelen transmitirse a la computadora personal empleando el puerto USB o el puerto Serie (utilizando el protocolo RS-232). Estas interfaces son de adopción masiva debido a la arquitectura abierta de las computadoras modernas.
- **LXI (LAN eXtensions for Instrumentation) / Ethernet:** Permite la integración del instrumento en redes de área local, facilitando tasas de transferencia de datos significativamente mayores y posibilitando el control y monitoreo remoto a largas distancias.

3.2.1 Protocolo VISA

La diversidad de buses físicos (GPIB, RS-232, USB, Ethernet) presenta un desafío a nivel de desarrollo de software, ya que tradicionalmente cada puerto requería llamadas al sistema y controladores (*drivers*) de bajo nivel completamente distintos. Para mitigar este problema, la industria adoptó la Arquitectura de Software para Instrumentos Virtuales (VISA, *Virtual Instrument Software Architecture*), que funciona como una capa de abstracción de Entrada/Salida, cuyo objetivo es proporcionar una interfaz de software, que encapsula y oculta los detalles de implementación del hardware subyacente. Lo que brinda las siguientes ventajas en el desarrollo de este trabajo:

- **Independencia del bus de hardware:** VISA permite que el software de control envíe comandos al osciloscopio sin necesidad de especificar cómo se gestionan eléctricamente los bits. Si el diseño del sistema migra de una conexión USB a una LAN, el código fuente de control de nivel superior no requiere modificaciones sustanciales.
- **Integración con comandos SCPI:** VISA actúa como el canal de transporte para el protocolo de Comandos Estándar para Instrumentos Programables (SCPI, *Standard Commands for Programmable Instruments*), que es un lenguaje basado en cadenas de texto ASCII estandarizadas (por ejemplo, `TIMEbase:SCALE?`) y los

instrumentos interpretan para ajustar sus parámetros como escala de tiempo, resolución vertical, o para entregar el contenido de su memoria hacia la computadora.

- **Gestión de recursos:** La arquitectura asigna descriptores unificados a cada instrumento detectado, centralizando la inicialización, la lectura de las muestras adquiridas, la escritura de comandos de configuración y el cierre de las sesiones de comunicación.

Los descriptores de recursos VISA por lo general, tienen la siguiente gramática estructural:

Interfaz[Índice]::Dirección_del_Dispositivo::[Recurso]

- **Interfaz:** Indica el tipo de controlador del puerto.
- **Índice:** Permite diferenciar numéricamente (por ejemplo, el 0 en USB0) cuando existen múltiples tarjetas controladoras del mismo tipo instaladas en la misma computadora (por ejemplo, dos placas PCI para GPIB distintas). Si se omite, VISA asume el valor cero por defecto.
- **Dirección del Dispositivo:** Identificador o ruta simbólica asignada para ubicar de forma unívoca el dispositivo.
- **Recurso:**
 - **INSTR:** La sesión se gestiona utilizando los mecanismos de VISA completos, permitiendo por ejemplo, el uso de comandos de aserción de eventos y de lectura de estado.
 - **SOCKET:** VISA actuará simplemente como un puente datos de bajo nivel sin enviar comandos de control propios del protocolo de instrumentación.

A continuación, se presentan ejemplos de descriptores de recursos VISA para cada tipo de interfaz física comúnmente utilizada en la adquisición de señales:

Tabla 3.1: Ejemplo de descriptor para ASRL (Asynchronous Serial Line)

ASRL1::INSTR	
ASRL1	Controlador lógico RS-232 o UART. Mapeado por el sistema al puerto físico (ej. COM1)

Tabla 3.2: Ejemplo de descriptor para USB (Universal Serial Bus)

USB0::0x0957::0x17A6::MY503::INSTR	
USB0	Controlador lógico USB.
0x0957	ID del fabricante.
0x17A6	ID del producto.
MY503	Número de serie único.

Tabla 3.3: Ejemplo de descriptor para GPIB (General Purpose Interface Bus)

GPIB0::14::INSTR	
GPIB0	Controlador lógico GPIB.
14	Dirección primaria. Rango de 0 a 30.

Tabla 3.4: Ejemplo de descriptor para TCPIP (LXI - VXI-11)

TCPIP0::192.168.1.100::inst0::INSTR	
TCPIP0	Controlador lógico TCPIP.
192.168.1.100	Dirección IPv4 del instrumento.
inst0	Nombre lógico de dispositivo.

Tabla 3.5: Ejemplo de descriptor para TCPIP (Socket - Raw)

TCPIP0::192.168.1.100::5025::SOCKET	
TCPIP0	Controlador lógico *Sockets* TCP.
192.168.1.100	Dirección IPv4 del instrumento.
5025	Puerto TCP específico.

Tabla 3.6: Ejemplo de descriptor para PXI (PCI eXtensions for Instrumentation)

PXI0::15::0::INSTR	
PXI0	Controlador lógico PXI.
15	Bus PCI
0	Posición física o lógica del dispositivo.

Capítulo 4

Procesamiento digital de señales

Este capítulo detalla los fundamentos teóricos del procesamiento digital de señales (DSP).

A lo largo de las siguientes secciones, se analizan las técnicas de filtrado digital orientadas a la mitigación de ruido sin alteración de la morfología del impulso, y los algoritmos de interpolación necesarios para la extracción paramétrica. Toda la metodología de procesamiento matemático y validación algorítmica se fundamenta en los lineamientos establecidos por la norma IEC 61083-2 para software utilizado en la evaluación de ensayos con impulsos de tensión y corriente.

4.1 Filtros digitales

Los filtros digitales son algoritmos matemáticos diseñados para modificar selectivamente las componente en frecuencia de una señal, permitiendo el paso de ciertas frecuencias y atenuando otras, tal como ocurre con los filtros analógicos constituidos por componentes electrónicos (resistencias, capacitores e inductores), ya que buscan simular su comportamiento. Estos se clasifican principalmente en dos grandes familias, FIR o IIR, según la duración de su respuesta al impulso.

4.1.1 Filtro de respuesta impulsional finita (FIR)

Un filtro FIR se caracteriza porque su respuesta al impulso decae a cero en un número finito de pasos. Se implementa mediante una estructura no recursiva (feedforward), lo que significa que la salida actual del filtro depende exclusivamente de los valores

presentes y pasados de la señal de entrada.

Su mayor ventaja es que son intrínsecamente estables, ya que carecen de lazos de retroalimentación que puedan causar oscilaciones.

Tienen desfasaje lineal, lo que asegura que todas las frecuencias de la señal poseen el mismo retraso temporal, logrando fidelidad de la forma de onda original.

Su principal limitación es la ineficiencia computacional. Ya que, para lograr un corte abrupto entre la banda de paso y la de atenuación, necesitan un orden muy elevado, lo cual exige mayor potencia de cálculo.

4.1.2 Filtro de respuesta impulsional infinita (IIR)

Un filtro IIR posee una respuesta al impulso que puede prolongarse indefinidamente en el tiempo. A diferencia de los FIR, estos filtros utilizan retroalimentación (feedback). Es decir, la salida actual se calcula utilizando las entradas actuales y pasadas, y además las salidas anteriores del propio filtro.

Destacan por su eficiencia computacional. Ya que, pueden lograr caídas de atenuación muy pronunciadas con un orden drásticamente menor que su equivalente en un filtro FIR, lo que los hace ideales para procesamiento muy rápidos en tiempo real o para sistemas con pocos recursos.

Su diseño es más complejo porque son condicionalmente estables, lo que implica que una mala ubicación de sus polos puede provocar que el filtro se vuelva inestable y oscile descontroladamente. Además, su respuesta de fase es no lineal, introduciendo una distorsión temporal en la señal y una distorsión de retardo de grupo en el dominio de la frecuencia.

4.1.3 Filtro de fase cero

Un filtro de fase cero es una técnica utilizada para eliminar el retraso temporal o desfase que los filtros suelen introducir en la salida de una señal. Esta técnica consiste en procesar los datos «hacia adelante y hacia atrás». Es decir, primero se filtra la señal original $x(n)$ en el dominio del tiempo, para crear una señal de salida $y(n)$, a la que se le invierten sus puntos de datos en el tiempo, se vuelven a pasar por el filtro, y por último se vuelven a invertir sus puntos de datos en el tiempo, dando como resultado, la salida final alineada en el tiempo con la señal de entrada original.

Dado que la información pasa dos veces por el filtro, se duplica la atenuación de las frecuencias filtradas. Lo que aumenta el costo computacional, ya que el tiempo de cálculo necesario es el doble en comparación con un filtrado estándar.

Es una operación que se realiza fácilmente con señales ya digitalizadas (como en un software de análisis de datos), pero es más difícil de implementar en sistemas con señales analógicas en tiempo real.

4.2 Ajuste de curvas e interpolación

Dado que los datos adquiridos por el osciloscopio, por más avanzado que sea, no dejan de ser puntos discretos que representan una señal continua, que además poseen ruido electromagnético y errores de cuantización. Si bien todos estos fenómenos se pueden atenuar, no se pueden eliminar por completo, por lo tanto, es necesario aplicar técnicas de ajuste de curvas (regresión) para obtener la función que mejor modele los datos, e interpolación para luego extraer los parámetros característicos del impulso.

4.2.1 Ajuste de curvas (regresión)

El ajuste de curvas es un proceso matemático que consiste en encontrar la función matemática que mejor represente la tendencia general de una distribución de datos. Su objetivo es minimizar el error global de las predicciones, lo que significa que la curva pasa cerca de los puntos, pero no necesariamente toca ninguno de ellos. Este método es especialmente útil cuando los datos no son completamente exactos, presentan dispersión o contienen errores de medición, como ocurre con las señales adquiridas en ensayos eléctricos.

Ahora bien, el tipo de función a ajustar debe ser elegido cuidadosamente para que represente adecuadamente la morfología del impulso, ya que este método para su ejecución requiere que se indique una función de base, sin ella no se puede realizar el ajuste porque no se puede evaluar el error al no tener una función de referencia. En este sentido, la norma IEC 60060-1 establece que la onda debe ajustarse a una función de doble exponencial (Ecuación 4.1), para caracterizar adecuadamente el comportamiento del impulso.

$$u_d(t) = U \cdot \left(e^{-\frac{t-t_d}{\tau_1}} - e^{-\frac{t-t_d}{\tau_2}} \right) \quad (4.1)$$

Donde:

$u_d(t)$: función de tensión.

t : tiempo.

U , τ_1 , τ_2 y t_d : parámetros a encontrar mediante el ajuste.

Para encontrar los parámetros de la función de doble exponencial, se utiliza el **método de mínimos cuadrados**, que es ampliamente usado en ingeniería para ajustar

modelos a datos experimentales. Este método busca minimizar la suma de los cuadrados de los residuos (la distancia vertical entre cada punto de dato observado y el punto correspondiente en la curva calculada). Matemáticamente, esto se expresa como:

$$S_r = \sum_{i=1}^N (y_{i,medicion} - y_{i,modelo})^2 \quad (4.2)$$

Donde $y_{i,medicion}$ son los valores observados, $y_{i,modelo}$ son los valores calculados por la función de doble exponencial, y N es el número total de datos observados.

Se utilizan los errores al cuadrado (en lugar de sumar simplemente los errores o sus valores absolutos) para evitar que las discrepancias positivas y negativas se cancelen entre sí, y también para asegurar que haya una única solución matemática óptima sin darle un peso desmedido a valores atípicos extremos.

Tipos de regresión

La regresión se clasifica principalmente por la relación matemática que hay entre las variables y el modelo:

Regresión Lineal Simple: Los datos muestran una tendencia constante o proporcional, que se ajusta mediante una línea recta.

Regresión Polinomial: Los datos presentan curvatura, ajustándose con polinomios de grado 2 o superior. Aunque generan curvas, se consideran modelos «lineales por mínimos cuadrados» porque la relación respecto a sus coeficientes desconocidos sigue siendo lineal.

Regresión Múltiple: La variable dependiente está influenciada por dos o más variables independientes.

Regresión No Lineal: Se utiliza para modelos matemáticos complejos que no son lineales con respecto a sus coeficientes, tales como ecuaciones de crecimiento exponencial o potencias.

Algoritmos de regresión no lineal

Para resolver los problemas de regresión, especialmente los no lineales, existen diversas técnicas:

Linealización: Es una técnica donde se aplican transformaciones matemáticas (como aplicar logaritmos) a una ecuación no lineal para convertirla en una forma lineal, permitiendo resolverla con los métodos simples de mínimos cuadrados.

Métodos de optimización iterativa: Cuando la ecuación no se puede linealizar, se

usan algoritmos numéricos que parten de un valor inicial (semilla) y lo mejoran paso a paso.

Existen muchos algoritmos de optimización disponibles, cada uno con sus ventajas y desventajas, pero debido a que en este trabajo se aplica la recomendación de la norma IEC 60060-1, se describe el modelo más relevante, el **Algoritmo de Levenberg-Marquardt (LM)**. El cual a su vez es considerado el estándar de la industria para regresiones no lineales. Es una técnica híbrida donde utiliza un factor de amortiguamiento dinámico (λ), que regula la convergencia, cuando los parámetros están lejos de la solución óptima o si el paso falla y el error aumenta, λ crece obligando al algoritmo a comportarse como *Descenso de Gradiente*, de pasos pequeños, seguros y robusto. Mientras que, a medida que se acerca a la solución, y el error disminuye, λ se hace más pequeño de manera que el algoritmo aproveche la velocidad de *Gauss-Newton*. Matemáticamente, esto se expresa como:

$$w_{i+1} = w_i - (J^T J + \lambda \text{diag}(J^T J))^{-1} (J^T e) \quad (4.3)$$

Donde:

w_{i+1} : vector de parámetros actualizado.

w_i : vector de parámetros actual.

λ : factor de amortiguamiento.

e : vector de residuos o errores.

J : matriz Jacobiana, formada por las primeras derivadas parciales de los errores respecto a los parámetros del modelo.

$J^T J$: matriz de Hessiana aproximada, compuesta por las segundas derivadas, que modelan la curvatura general de la función de error.

$\text{diag}(J^T J)$: matriz diagonal. Escala cada parámetro según la curvatura local de la función.

$J^T e$: vector gradiente de la función error. Indica la dirección de máxima pendiente.

Si $\lambda \rightarrow 0$, el algoritmo se comporta como el **Método de Gauss-Newton** donde asume que la función de error es localmente cuadrática y utiliza expansiones de series de Taylor para aproximarse al mínimo de forma muy rápida, pero puede ser inestable y fallar si los valores iniciales elegidos están muy lejos del óptimo. Resultando:

$$w_{i+1} = w_i - (J^T J)^{-1} (J^T e) \quad (4.4)$$

Si $\lambda \rightarrow \infty$, se comporta como el **Descenso del gradiente**, que actualiza los coeficientes moviéndose en la dirección donde el error desciende más rápido. Es muy robusto, es difícil que diverja, pero su convergencia puede ser muy lenta.

$$w_{i+1} = w_i - \alpha (J^T e) \quad (4.5)$$

Donde:

α : tamaño de paso de avance de la iteración.

En el metodo de descenso del gradiente α es un escalar, pero en el algoritmo de Levenberg-Marquardt, resulta una matriz diagonal que escala cada parámetro de forma independiente, por lo tanto:

$$\alpha = \text{diag}(J^T J) \quad (4.6)$$

4.2.2 Interpolación

Debido a que, como se dijo antes, los datos adquiridos por el osciloscopio son puntos discretos que representan una señal continua, es necesario aplicar técnicas de interpolación para obtener valores intermedios entre los puntos muestreados. Esto es necesario para encontrar puntos como U_{30} y U_{90} , que pueden no coincidir exactamente con los datos de datos adquiridos.

La interpolación polinomial es un proceso matemático que consiste en determinar el único polinomio de grado n que pasa por los $n + 1$ puntos de datos conocidos, permitiendo así calcular los valores intermedios. Existen varios métodos de interpolación, cada uno con sus propias características y aplicaciones, entre los más comunes se encuentran:

Interpolación Lineal: Es el método más simple, que conecta dos puntos adyacentes mediante un polinomio de primer grado (una recta). Es rápido pero puede introducir errores significativos si la función subyacente es no lineal con gran curvatura, o si los puntos están muy alejados. Matemáticamente, se puede expresar como:

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0) \quad (4.7)$$

Donde:

$f_1(x)$: polinomio de interpolación de primer grado.

$f(x_0)$: punto de origen.

$\frac{f(x_1) - f(x_0)}{x_1 - x_0}$: pendiente de la recta.

$(x - x_0)$: distancia desde el punto de origen hasta el objetivo.

Interpolación Polinomial: Utiliza un polinomio de grado n para pasar por $n + 1$ puntos. Puede proporcionar una aproximación más precisa, pero es propenso a oscilaciones si el grado del polinomio es demasiado alto o si los puntos están mal distribuidos. Matemáticamente puede generalizarse como:

$$f_n(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) + \dots + b_n(x - x_0)(x - x_1) \dots (x - x_{n-1})$$

Donde los coeficientes b_i son las diferencias divididas finitas evaluadas a partir de los

datos.

$$\begin{aligned}b_0 &= f(x_0) \\b_1 &= f[x_1, x_0] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} \\b_2 &= f[x_2, x_1, x_0] = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0} \\b_n &= f[x_n, \dots, x_1, x_0] = \frac{f[x_n, \dots, x_1] - f[x_{n-1}, \dots, x_0]}{x_n - x_0}\end{aligned}$$

Lo siguiente va en el marco metodológico.

Durante la captura de impulsos de alta tensión, la señal suele contaminarse con interferencia electromagnética (EMI) de alta frecuencia originada por el disparo de los espinterómetros (utilizados como interruptores de alta tensión), y el ruido del propio sistema de medida. La aplicación de filtros digitales post-adquisición es fundamental para separar la señal de interés del ruido, pero requiere cuidado de no distorsionar la señal real.

La norma IEC 60060-1 recomienda usar filtros de fase cero cuya respuesta en frecuencia sea igual a la función de tensión de prueba Ecuación 2.1.

La norma IEC 60060-1 presenta un ejemplo de un filtro de respuesta impulsional infinita (IIR) de fase cero. Aunque también permite utilizar otros filtros, como los de respuesta impulsional finita (FIR), así como otros softwares comerciales.

En este enfoque, la atenuación del filtro es la mitad de la necesaria, pero los datos se procesan dos veces, primero en sentido directo y luego en sentido inverso. De este modo, el filtrado tiene una salida que coincide con la función de tensión de ensayo con un error de amplitud y un desfase prácticamente nulos.

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n - k]$$

En contraste, el uso de filtros IIR (Infinite Impulse Response) como el Butterworth debe ser manejado con extrema cautela. A pesar de su excelente selectividad de frecuencia, su respuesta de fase no lineal introduce distorsión de retardo de grupo, lo cual puede deformar irreversiblemente la morfología del transitorio si no se aplican técnicas de filtrado bidireccional (zero-phase filtering).

Capítulo 5

Arquitectura del software y entorno de desarrollo

Este capítulo detalla el diseño estructural del software desarrollado para la adquisición y análisis de ondas de impulso tipo rayo ($1, 2/50 \mu s$). Se describe el patrón de diseño implementado para garantizar la escalabilidad y el desacople de los componentes, el flujo de datos desde el osciloscopio digital hasta la generación de reportes, y se presenta la justificación técnica del *stack* de desarrollo seleccionado en contraste con otras alternativas de la industria. Finalmente, se definen los lineamientos de gestión del entorno de desarrollo y el control de versiones aplicados durante el ciclo de vida del proyecto.

5.1 Patrones de arquitectura del software

Actualmente, para el desarrollo de software en general, se existen diversos patrones de diseño que ayudan a organizar el código, mejorar la mantenibilidad y facilitar la escalabilidad. Algunos de los patrones más comunes son:

Modelo-Vista-Controlador (MVC, *Model-View-Controller*): es un patrón de diseño de software que se utiliza para separar la lógica de presentación, la lógica de negocio (procesar los datos, realizar cálculos y tomar decisiones) y la gestión de eventos en una aplicación. En este patrón, el **Modelo** representa los datos y la lógica de negocio, la **Vista** se encarga de la presentación visual y la interacción con el usuario, y el **Controlador** actúa como intermediario entre el Modelo y la Vista, gestionando las entradas del usuario y actualizando la Vista en consecuencia.

Modelo-Vista-Presentador (MVP, *Model-View-Presenter*): es similar al MVC, pero con una separación más clara entre la lógica de presentación y la lógica de negocio. En este patrón, el **Presentador** se encarga de manejar la lógica de presentación y la interacción con el usuario, mientras que el **Modelo** sigue representando los datos y la lógica de negocio. La **Vista** se limita a mostrar los datos proporcionados por el Presentador y a enviar las acciones del usuario al Presentador.

Modelo-Vista-VistaModelo (MVVM, *Model-View-ViewModel*): es una evolución del MVP, el cual introduce el concepto de **ViewModel**, que actúa como un enlace entre el Modelo y la Vista. En este patrón, el ViewModel expone los datos y comandos que la Vista puede enlazar directamente, lo que facilita la separación de responsabilidades y mejora la testabilidad de la aplicación.

5.1.1 Arquitectura de software adoptada

El sistema se diseñó bajo el patrón arquitectónico Modelo-Vista-Presentador, con el objetivo de separar estrictamente la lógica de presentación (GUI), la lógica de negocio, y la gestión de eventos (presionar botones, ingresar texto o capturar una señal). Esta segregación es fundamental para cumplir con los requerimientos de validación algorítmica establecidos por la norma IEC 61083-2, permitiendo someter los módulos matemáticos a pruebas unitarias, utilizando los datos de calibración provistos en el software Generador de Datos de Prueba (TDG, *Test Data Generator*), incluido en la norma, sin dependencia de la interfaz gráfica.

Los componentes de la arquitectura se distribuyen de la siguiente manera:

- **Vista:** Maneja exclusivamente la representación visual del software y la interacción del usuario. Es generada con la herramienta gráfica Qt Designer para facilitar su desarrollo. Se mantiene completamente independiente de la lógica de adquisición y procesamiento, emitiendo señales (Qt Signals) que son capturadas por el presentador para desencadenar las acciones correspondientes.
- **Modelo:** Está compuesto por tres módulos, maneja el hardware mediante **GWInstekGDS1000AU**, tiene la lógica de persistencia en **FileManager**, y centraliza los cálculos en **ImpulseAnalyzer**, donde realiza el procesamiento de señales, la aplicación de filtros digitales y el ajuste de curvas para extraer los parámetros de la onda.
- **Presentador:** Implementado en la clase **MainController**. Actúa como orquestador conectando las señales emitidas por la Vista con los métodos receptores (*slots*) correspondientes para interactuar con el Modelo, y actualizando la interfaz gráfica con los resultados procesados.

5.1.2 Manejo de Concurrency y Flujo de Datos

Dado que la comunicación con el osciloscopio (GW Instek GDS-1102A-U) requiere operaciones de entrada/salida (I/O) bloqueantes —específicamente durante la espera del disparo (*trigger*) del impulso—, la arquitectura incorpora un enfoque multihilo.

Se implementó la clase `WaitWaveformThread` heredada de `QThread`. Este hilo secundario sondea el estado del osciloscopio en segundo plano. Cuando se detecta el disparo, el hilo emite una señal (`wave_detected`) que es capturada por el hilo principal para proceder con la descarga del bloque de datos. Este diseño previene el congelamiento (*freezing*) de la GUI.

El flujo de datos continuo se define en cuatro etapas secuenciales:

1. **Adquisición:** El osciloscopio digitaliza la señal analógica. El bloque de datos binario es transferido a la PC vía USB (PyVISA), desempaquetado y convertido a un arreglo considerando los factores de atenuación del sistema de medición.
2. **Procesamiento:** En `LightningImpulseAnalyzer` se procesa el arreglo de tensión bruto, aplicando la rutina indicada en el Anexo B de la norma IEC 60060-1, para obtener los parámetros característicos de la onda.
3. **Visualización:** Los arreglos resultantes se renderizan en pantalla.
4. **Persistencia:** Los datos del ensayo, las condiciones ambientales y los vectores de tensión (brutos y procesados) se empaquetan y almacenan en un archivo HDF5, garantizando la trazabilidad de los datos.

Parte II

MARCO METODOLÓGICO

Capítulo 6

Implementación del Software

6.1 Arquitectura general

El diseño estructural del software se fundamenta en una separación de responsabilidades, que permite desarrollar y evaluar de forma independiente cada parte del sistema. Con este objetivo, se adoptó el patrón de diseño arquitectónico MVP, el cual divide el sistema en tres capas lógicas bien definidas: la Vista, el Modelo y el Presentador. Esta organización asegura el desacoplamiento de componentes, facilitando la escalabilidad del sistema ante la incorporación de otros modelos de osciloscopios o métodos adicionales de análisis.

La división de tareas facilita la validación algorítmica y control de la incertidumbre matemática especificados en la norma [1], permitiendo someter los algoritmos a ensayos automatizados, aislados de cualquier interfaz gráfica de usuario.

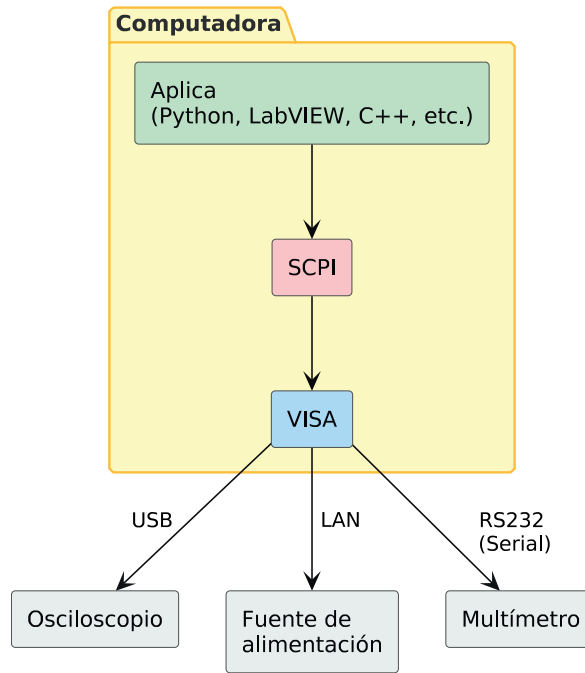


Figura 6.1: Diagrama de bloque de la arquitectura.

6.1.1 Capa de Presentación: Vista

La **Vista** maneja de forma exclusiva la representación visual del sistema y la captura de las interacciones del operador. Ya que, solo declara estáticamente la disposición y propiedades de los componentes gráficos.

Esta capa se caracteriza por no tener referencias directas hacia la lógica de adquisición física o los algoritmos de procesamiento digital de señales. Ante cualquier interacción física del usuario, como el accionamiento de botones o la inserción de datos, la Vista emite eventos en forma de *señales de Qt*, que son interceptadas por el Presentador para desencadenar el flujo correspondiente.

6.1.2 Capa de Lógica y Datos: Modelo

El **Modelo** representa el núcleo funcional del software, abarcando el control del instrumental de laboratorio, la lógica de cálculo metrológico y la administración de la persistencia de datos.

Esta capa se subdivide en tres submódulos independientes especializados:

1. **Abstracción de Hardware:** Implementada en el módulo **GWInstekGDS1000AU**. Encapsula todas las operaciones de control, configuración de canales y adquisición de datos de la memoria del osciloscopio, mediante comandos SCPI, usando el protocolo de comunicación PyVISA.

2. **Persistencia y Gestión de Archivos:** Administrada por el módulo **FileManager**. Maneja la creación y el acceso al sistema de archivos. Su función principal es el almacenamiento en bases de datos utilizando el formato HDF5. Asimismo, gestiona la exportación de resultados a hojas de cálculo tipo CSV o XLSX.
3. **Procesamiento Digital de Señales:** Llevado a cabo por el módulo matemático **LightningImpulseAnalyzer**. Constituye el motor matemático del sistema. Es responsable de acondicionar la señal y extraer sus parámetros característicos según la norma [2].

6.1.3 Capa de Coordinación: Presentador

El **Presentador**, es implementado en el módulo **MainPresenter**, y actúa como coordinador de la arquitectura MVP. Tiene como responsabilidad orquestar la interacción entre la interfaz visual y los motores lógico-físicos.

6.2 Controlador de *hardware*

En esta sección se expone la implementación controlador del osciloscopio, cuya función general es abstraer las operaciones de bajo nivel, permitiendo desacoplar la comunicación con el dispositivo del resto del sistema.

6.2.1 Interfaz de comunicación y capa de abstracción de hardware

A nivel de *software*, para evitar el acoplamiento directo del código de la aplicación con las llamadas al sistema de bajo nivel (capa física de transmisión), se implementa una Capa de Abstracción de Hardware (HAL, *Hardware Abstraction Layer*), bajo la Arquitectura VISA. Esta capa de abstracción permite que los comandos de consulta (*queries*) y configuración (*writes*) permanezcan independientes de la interfaz de hardware utilizada (sea USB, GPIB, Ethernet o RS-232), permitiendo actualizaciones de la instrumentación sin necesidad de modificar el código del presentador.

El controlador inicializa el administrador de recursos, y ejecuta un escaneo automático para buscar algún dispositivo conectado, que tenga un descriptor de recurso lógico VISA, como el siguiente:

ASRL1 : INSTR

Si encuentra al menos un dispositivo direccionable, se conecta al mismo. La comunicación inicia instanciando el administrador de recursos VISA de Python (PyVISA),

acompañado de la biblioteca `pyvisa-py` para asegurar un entorno multiplataforma sin depender de bibliotecas nativas propietarias.

Sobre el canal de transporte establecido con VISA, el software envía comandos SCPI, que debe cumplir con el siguiente formato indicado en el manual del fabricante [3]:

- **No sensibilidad a mayúsculas:** Puede usarse mayúsculas y minúsculas.
- **Separación de parámetros:** Al configurar parámetros, debe separarse el comando y su argumento numérico, mediante un carácter de espacio en blanco simple.
- **Finalizador de mensaje:** Cada instrucción enviada al osciloscopio debe finalizar con el carácter delimitador de fin de mensaje *Line Feed* (**LF**, en hexadecimal **0x0A** o a la secuencia de escape **\n**), para que el instrumento proceda a interpretar y ejecutar la orden.

Para gestionar la configuración del osciloscopio de manera robusta, se implementa una arquitectura, basada en métodos de consulta (*getters*) y configuración (*setters*), que encapsulan los comandos SCPI.

Los métodos de consulta usan la instrucción `.query()`, reciben una cadena de caracteres, que deben convertir al tipo de dato correspondiente (`float`, `int` o `string`) y la retornan al presentador. En tanto que, los métodos de configuración emplean la instrucción `.write()`, sin esperar ninguna respuesta del instrumento.

Este flujo se protege con una rutina de control de excepciones que, ante fallos críticos en el bus o pérdidas de sincronismo, aborta la operación y libera los recursos del sistema de forma segura, ya que de lo contrario impediría la reconexión posterior del instrumento, debiendo reiniciar la computadora o el osciloscopio.

A continuación, se ejemplifican los métodos de consulta y configuración de la escala vertical de un canal.

```
# Método de consulta.
def get_channel_scale(channel):
    scale = dso.query(f':channel{channel}:scale?')
    v_scale = float(scale)
    return v_scale

# Método de configuración.
def set_channel_scale(channel, value):
    dso.write(f':channel{channel}:scale {value}')
```

Por otro lado, a nivel físico, la conexión se realiza a través de un puerto USB 2.0 tipo B (esclavo) ubicado en el panel posterior del instrumento, y un puerto USB tipo A (maestro) de la computadora. Aunque, si bien el equipo se conecta a través de un puerto USB, la computadora lo detecta como un dispositivo de comunicación serie vir-

tual¹, bajo la Clase de Dispositivo de Comunicación con Modelo de Control Abstracto (CDC-ACM, *Communication Device Class - Abstract Control Model*), que aparece en el administrador de dispositivos con el siguiente nombre [3]:

COM1

Donde COM1 identifica el número de puerto serie virtual asignado por el sistema operativo.

6.2.2 Sistema de Adquisición y acondicionamiento

La adquisición del impulso de alta tensión desde el generador Marx, hasta el almacenamiento en la computadora, se estructuran de la siguiente forma:

1. Acondicionamiento analógico.
2. Captura de impulso
3. Digitalización.
4. Desempaquetado de datos.
5. Reconstrucción y escalamiento de la onda.

Acondicionamiento analógico

La señal eléctrica, debe ser adaptada en amplitud antes de ingresar al osciloscopio, para no dañarlo.

Para el canal de tensión (CH1), la señal se deriva utilizando un divisor de tensión resistivo, acoplado en cascada serie con un atenuador coaxial, por lo que el factor de atenuación equivalente resulta:

$$A_{CH1} = \text{Divisor} \times \text{Atenuador} \quad (6.1)$$

Para el canal de corriente (CH2), la descarga atraviesa una resistencia shunt conectada en serie con el elemento bajo ensayo, de la cual se toma su tensión a bornes, acoplada a un atenuador que reduce su magnitud a niveles adecuados.

$$A_{CH2} = R_{\text{Shunt}} \times \text{Atenuador} \quad (6.2)$$

Ambas señales llegan al instrumento por medio de cables coaxiales de 50Ω , para reducir acoplamientos electromagnéticos parásitos (EMI).

¹El controlador (*driver*) USB, está disponible en la página del fabricante [4].

Captura de impulso

La captura de impulsos ($1.2/50 \mu\text{s}$) requiere que el osciloscopio opere en modo de disparo único (*Single Trigger*), dado que es un transitorio no periódico.

Cuando el operador inicia el modo de captura desde la GUI, el presentador crea y ejecuta un subproceso asíncrono, que mediante el controlador de *hardware*, realiza un sondeo periódico (*polling*) al osciloscopio, para saber el estado de disparo del mismo.

Al detectar el impulso finaliza el bucle de *polling* y emite una señal, que es capturada en el hilo de ejecución principal por un receptor (*slot*) del presentador, y finaliza el subproceso.

Como el instante de descarga es indeterminado, se implementa un hilo de ejecución secundario asíncrono, dedicado a consultar el estado de disparo, ya que si se ejecutara en el hilo principal, sería una operación bloqueante, que congele la aplicación e inhabilite la GUI. Por lo que, así se mantiene siempre reactivo el hilo principal de eventos gráficos, permitiendo al usuario manipular oscilogramas previos, cargar campos o cualquier otra acción, mientras el sistema espera la captura del impulso.

Digitalización

Las señales eléctricas que ingresan a los bornes analógicos del osciloscopio, se muestran a una tasa que depende de la base de tiempo utilizada. Después, el convertidor analógico-digital (ADC) realiza la cuantización en amplitud, dividiendo el rango dinámico vertical en $2^8 = 256$ niveles de tensión discretos (8 bits). A continuación, se graban en la memoria del instrumento.

Desempaquetado de datos

Luego de que el osciloscopio digitalice la señal, el presentador ordena la descarga de los datos brutos (*RAW*) de los canales habilitados, y recibe un bloque de datos, compuestos por el encabezado y el vector de datos de la onda, que cumple con la siguiente estructura²³:

A B C D E

#: [1 byte] Caracter ASCII que indica el inicio de la cabecera.

²Debido a discrepancias entre la estructura de datos obtenida del instrumento y el manual del fabricante, se adoptó un script provisto por el soporte técnico del fabricante.

³El manual de programación de la serie GDS-1000B complementa los detalles para manipular esta estructura.

- A: [1 byte] Dígito ASCII que especifica la cantidad de caracteres que componen el campo B. Para adquisiciones estándar toma el valor ASCII '4', mientras que para adquisiciones de memoria extendida toma el valor '7'.
- B: [4 o 7 bytes] Cadena de caracteres ASCII que representa el tamaño total en bytes del bloque de datos que le sigue (C+D+E). Para adquisiciones estándar toma el valor ASCII '8008', mientras que para adquisiciones de memoria extendida puede tomar los valores '2000008' o '4000008'.
- C: [4 bytes] Intervalo de tiempo (Δt) entre dos muestras consecutivas de la señal, en formato de punto flotante de precisión simple según IEEE-754 (32 bits), en orden *little-endian* (se envía primero el byte menos significativo):

$$\Delta t = t[n + 1] - t[n]$$

- D: [4 bytes] Bloque de datos no utilizado.
- E: [8×10^3 , 2×10^6 o 4×10^6 bytes] Vector de datos de la onda, cuya longitud es definida en el campo B. Cada valor entero (base 10) es expresado con 2 bytes (16 bits), y transmitidos en orden *big-endian* (se envía primero el byte más significativo).

Una vez finalizada la descarga de datos, el presentador ordena al controlador de *hardware* que decodifique cada bloque acorde a su formato, considerando la estructura recién mencionada.

Reconstrucción y escalamiento de la onda

Con el vector de valores enteros brutos decodificado, primero se aplica la ecuación (6.3) para reconstruir la forma de onda que llegó a bornes del osciloscopio, considerando la escala vertical configurada al momento de la captura, y el Factor AD = 25 definido en el manual de programación del fabricante [3]:

$$\text{Onda sin escalar}[n] = \text{Datos brutos}[n] \times \frac{V/\text{div}}{\text{Factor AD}} \quad (6.3)$$

A continuación, para obtener las magnitudes reales aplicadas al objeto bajo ensayo, se multiplica escalarmente los vectores de tensión y corriente, por sus respectivos factores de atenuación, cargados por el usuario en los campos pertinentes de la ventana, aplicando la siguiente ecuación:

$$\text{Onda escalada}[n] = \text{Onda sin escalar}[n] \times A_{\text{CH}<\text{X}>} \quad (6.4)$$

Los vectores resultantes de tensión (en volts, V) y de corriente (en amperes, A), se transfieren al módulo de procesamiento matemático para determinar sus parámetros normativos: tiempo de frente T_1 , tiempo de cola T_2 , valor de cresta U_t y sobreimpulso OS, conforme a los requerimientos de la norma [2].

6.3 Motor de Cálculo: Procesamiento de Señales y Ajuste de Curvas

Este componente constituye el núcleo matemático del software, y es responsable de obtener los parámetros normativos a partir de los arreglos temporales de tensiones discretas adquiridas por el *hardware*.

El motor de cálculo ha sido diseñado exclusivamente para cumplir con las normas IEC 60060-1 [2] e IEC 61083-2-1 [1], aplicando criterios de modularidad, que permiten su validación automatizada e independiente del resto de la aplicación.

6.3.1 Flujo algorítmico de cálculo

Este proceso se divide en tres fases operativas: preprocesamiento común, clasificación dinámica y reconstrucción de la señal.

Fase I: Acondicionamiento Inicial

Esta etapa es común a todos los registros adquiridos.

1. **Eliminación de offset:** Se calcula el valor de tensión continua (V_{offset}) promediando estadísticamente una ventana de datos adquiridos ($N_{\text{pretrigger}}$) en el instante previo al disparo (“pre-trigger”), donde la curva registrada ($U_e[n]$) es nula.

$$V_{\text{offset}} = \frac{1}{N_{\text{pretrigger}}} \sum_{n=1}^{N_{\text{pretrigger}}} U_e[n] \quad (6.5)$$

El vector de tensión corregida se calcula como:

$$U_{\text{corregida}}[n] = U_e[n] - V_{\text{offset}} \quad (6.6)$$

2. **Normalización de polaridad:** Dado que los impulsos pueden ser de polaridad positiva o negativa, el algoritmo detecta el sentido del impulso evaluando el signo de la muestra extrema:

$$\text{polaridad} = \text{sign}(\arg \max |U_{\text{corregida}}[n]|) \quad (6.7)$$

El vector de tensión corregida se multiplica por este término para simplificar el resto del procesamiento, asegurando que el análisis sea exclusivamente en el semiplano positivo:

$$U_{\text{abs}}[n] = \text{polaridad} \cdot U_{\text{corregida}}[n] \quad (6.8)$$

3. **Normalización unitaria:** Se normaliza la amplitud al dominio unitario respecto

a su valor de cresta.

$$U_{\text{norm}}[n] = \frac{U_{\text{abs}}[n]}{\max(U_{\text{abs}}[n])} \quad (6.9)$$

Fase II: Bifurcación de análisis e identificación de parámetros base

En este punto, el flujo se divide según si debe procesar un impulso de referencia (pleno) o un impulso de ensayo, que puede ser pleno o cortado.

Caso A: Análisis de impulso de referencia (pleno)

Se asume que la onda es plena y se procede al ajuste paramétrico.

4. **Segmentación de datos:** Se selecciona una ventana de datos en el intervalo comprendido entre el 20 % en el flanco de subida y el 40 % en la cola de descarga del impulso.
5. **Ajuste de curva base:** Se realiza un ajuste paramétrico no lineal sobre la ventana de muestras, utilizando el algoritmo de **Levenberg-Marquardt**, teniendo como modelo de estimación la función de doble exponencial:

$$u_d[n] = U \cdot \left(e^{-\frac{t-t_d}{\tau_1}} - e^{-\frac{t-t_d}{\tau_2}} \right) \quad (6.10)$$

Donde los coeficientes desconocidos a determinar son:

- U el parámetro de amplitud escalar.
- τ_1 la constante de tiempo de descarga.
- τ_2 la constante de tiempo de carga.
- t_d el retardo temporal del origen virtual.

Caso B: Análisis de ensayo (LI vs. LIC)

Si evalúa una onda de ensayo, se inyectan los parámetros de la onda de referencia procesada en el **Caso A**, y se ejecuta el siguiente análisis:

6. **Cálculo del desfase temporal:** El corrimiento temporal, t_L , entre el frente de onda del último impulso con el de referencia, se calcula como el promedio de las diferencias de los instantes de tiempo, correspondientes al 30 %, 50 % y 80 % de U_t respectivamente, de ambos impulsos.

$$t_L = \frac{(t_{30,\text{ref}} - t_{30,\text{i}}) + (t_{50,\text{ref}} - t_{50,\text{i}}) + (t_{80,\text{ref}} - t_{80,\text{i}})}{3} \quad (6.11)$$

7. **Alineación del eje temporal:** Se obtiene el vector temporal alineado del impulso de ensayo, $t_{\text{aligned}}[n]$, partiendo de su vector temporal original $t[n]$, al que se lo desplaza la cantidad t_L .

$$t_{\text{aligned}}[n] = t[n] + t_L \quad (6.12)$$

8. **Clasificación del impulso:** Se calcula la diferencia absoluta punto a punto entre

el impulso de ensayo y el de referencia, usando sus vectores normalizados en amplitud.

$$|U_{\text{norm}, i}[n] - U_{\text{norm}, \text{ref}}[n]| > \text{umbral} \quad (6.13)$$

En caso de que la desviación no supere el umbral, la onda es clasificada como **impulso pleno (LI)** y el flujo se dirige al **Caso A**.

Si la desviación supera el umbral, la onda se clasifica como **impulso cortado (LIC)** y se procede a analizar el colapso de la onda.

9. **Cálculo del factor de escala:** Se calcula como el promedio del intervalo comprendido entre el 30 % y 80 % de $U_{\text{corregida}}[n]$ del frente de la onda de ensayo y la de referencia.

$$\overline{U_i} = \frac{1}{N} \sum_{n=k_{30\%}}^{k_{80\%}} U_i[n] \quad (6.14)$$

$$\overline{U_{\text{ref}}} = \frac{1}{N} \sum_{n=k_{30\%}}^{k_{80\%}} U_{\text{ref}}[n] \quad (6.15)$$

$$E = \frac{\overline{U_i}}{\overline{U_{\text{ref}}}} \quad (6.16)$$

10. **Escalado de la curva base:** Debido a que la descarga disruptiva hace colapsar el ajuste por el corte abrupto de la señal, se sintetiza la curva base de la onda de referencia $U_{b, \text{ref}}[n]$, y se escala en amplitud con el factor de escala E .

$$U_{b, i}[n] = E \cdot U_{b, \text{ref}}[n] \quad (6.17)$$

11. **Tiempo de corte:** Se analiza la cola del impulso, minimizando la derivada primera para localizar el sector de máxima pendiente de caída:

$$n_{\text{abrupto}} = \arg \min \left(\frac{dU_{\text{cola}}[n]}{dt} \right) \quad (6.18)$$

El inicio del colapso se encuentra evaluando el punto de inflexión local mediante la derivada segunda:

$$n_{\text{inflexion}} = \arg \min \left(\frac{d^2 U_{\text{cola}}[n]}{d^2 t} \right) \quad (6.19)$$

Con ese índice se obtiene la tensión del colapso:

$$U_{\text{colapso}} = U_{\text{corregida}}[n_{\text{pico}} + n_{\text{inflexion}}] \quad (6.20)$$

Se calculan los instantes correspondientes al 70 % y al 10 % de U_{colapso} sobre el flanco descendente, para extrapolar linealmente el instante de corte, para el ni-

vel de U_{colapso} :

$$t_c = t_{70} + (U_{\text{colapso}} - U_{70}) \left(\frac{t_{10} - t_{70}}{U_{10} - U_{70}} \right) \quad (6.21)$$

Finalmente, se calcula el tiempo de corte respecto al origen virtual:

$$T_c = t_c - O_1 \quad (6.22)$$

Fase III: Convergencia, Filtrado y Parámetros Característicos

Independientemente de la ruta ejecutada en la Fase II, el algoritmo converge obteniendo una curva registrada y una curva base (ajustada numéricamente o escalada analíticamente), para avanzar a la secuencia final:

12. **Cálculo de curva residual:** Se obtiene como la diferencia algebraica entre la curva corregida ($U_{\text{corregida}}[n]$) y la curva base ($U_b[n]$) construida con los parámetros obtenidos del ajuste:

$$R[n] = U_{\text{corregida}}[n] - U_b[n] \quad (6.23)$$

13. **Filtrado digital de fase cero:** Se obtiene la curva residual filtrada ($R_f[n]$), aplicando un filtro digital IIR a la curva residual ($R[n]$). Para reducir el ruido de muy alta frecuencia relacionado a la interferencia electromagnética, sin perder oscilaciones propias del ensayo.
14. **Curva de tensión de ensayo:** Se sintetiza la curva de ensayo ($U_t[n]$) sumando algebraicamente la curva base analítica con la residual filtrada:

$$U_t[n] = U_b[n] + R_f[n] \quad (6.24)$$

15. **Cálculo de parámetros característicos:** A partir de la curva de tensión de ensayo $U_t[n]$, se calculan los parámetros normativos. Dado que la señal procesada es de tiempo discreto, hay valores analíticos que pueden no corresponder con las muestras obtenidas, por lo que para encontrarlos se realiza una interpolación lineal.

- **Valor de cresta (U_t):** Es la amplitud máxima de la curva de tensión de ensayo $U_t[n]$:

$$U_t = \max(U_t[n]) \quad (6.25)$$

- **Tiempo de frente (T_1):** Se calcula como el cociente del intervalo temporal transcurrido entre los instantes t_{30} y t_{90} (correspondientes al 30 % y 90 % de U_t respectivamente), dividido por 0,6:

$$T_1 = \frac{t_{90} - t_{30}}{0,6} \quad (6.26)$$

- **Origen virtual (O_1):** El inicio ficticio de la descarga eléctrica se calcula:

$$O_1 = t_{30} - 0,5 \cdot (t_{90} - t_{30}) \quad (6.27)$$

- **Tiempo de cola (T_2):** El intervalo temporal entre el origen virtual O_1 y el instante t_{50} , correspondiente al 50 % de U_t en el flanco descendente de la onda, se calcula:

$$T_2 = t_{50} - O_1 \quad (6.28)$$

- **Sobreimpulso relativo (OS%):** Se calcula como la diferencia entre el valor extremo de la curva registrada (U_e):

$$U_e = \max(U_e[n]) \quad (6.29)$$

y el valor máximo de la curva base (U_b):

$$U_b = \max(U_b[n]) \quad (6.30)$$

expresado en porcentaje:

$$OS\% = \frac{U_e - U_b}{U_e} \cdot 100 \quad (6.31)$$

6.3.2 Precisión del ajuste de curva

Debido a la naturaleza de la forma de onda estudiada, la cantidad de puntos es significativamente mayor en la zona de la cola que en el frente de la onda (donde los gradientes de tensión son mayores y transcurren en menor tiempo), el ajuste de mínimos cuadrados ordinario en algunos casos puede converger erróneamente, ya que el optimizador tiende de forma natural a minimizar el error residual en la cola, provocando desviaciones considerables en el frente.

Para minimizar el error en el frente de la onda, se introduce el vector de varianza estimada σ_i , con distintos pesos a través del parámetro `sigma_fit`, que se parametriza de la siguiente manera:

$$\sigma_i = \begin{cases} \text{sigma_fit} & \text{si } i \leq \text{idx_peak} + 5 \\ 1.0 & \text{si } i > \text{idx_peak} + 5 \end{cases} \quad (6.32)$$

Al configurar un valor de `sigma_fit` ≤ 1 , el algoritmo LM penaliza con mayor severidad los residuos matemáticos en el frente de la onda, mejorando la precisión en esta zona.

6.3.3 Cálculo del Filtro de fase cero

El filtro digital de respuesta al impulso infinita (IIR), se diseña tal que su respuesta en frecuencia sea la función de la tensión de ensayo, establecida en la norma IEC 60060-1:

$$k(f) = \frac{1}{1 + 2,2 \cdot f^2} \quad (6.33)$$

Donde f se expresa en MHz.

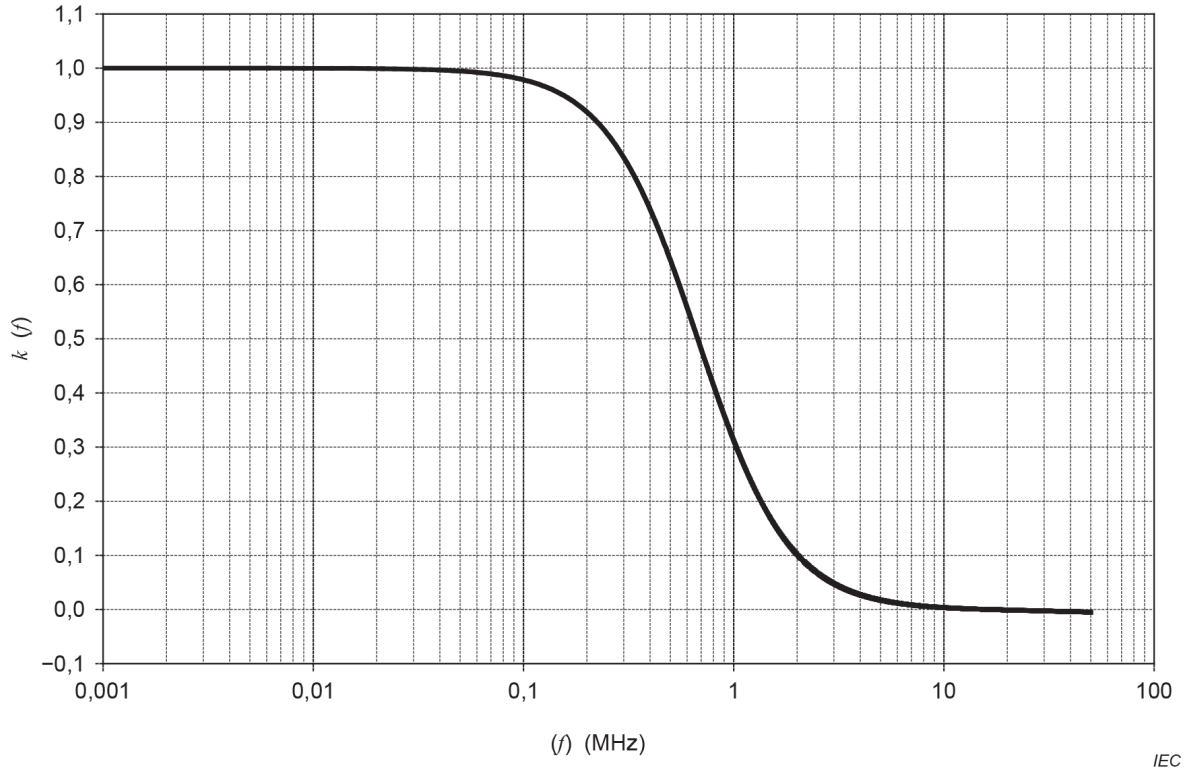


Figura 6.2: Función tensión de ensayo.

La señal se procesa primero en sentido directo y después en sentido reverso, para anular el desplazamiento de fase introducido por el filtro, ejecutando la siguiente ecuación de diferencias recursiva:

$$y[i] = b_0 \cdot x[i] + b_1 \cdot x[i - 1] + a_1 \cdot y[i - 1] \quad (6.34)$$

Los coeficientes del filtro directo se calculan mediante:

$$b_0 = b_1 = \frac{c}{1 + c}$$

$$a_1 = \frac{1 - c}{1 + c}$$

El parámetro c se calcula:

$$c = \tan \left(\frac{\pi \cdot T_s}{\sqrt{d}} \right) \quad (6.35)$$

Donde:

T_s es el periodo de muestreo del impulso.

La constante $d = 2,2 \times 10^{-12}$ está definida en el Anexo D.2 de IEC 60060-1.

=====

6.4 Elección del stack tecnológico

Para desarrollar el software de control del osciloscopio y procesamiento de los datos, se analizaron tres opciones que se consideran las más adecuadas para este trabajo: Python, C++ y LabVIEW. Estas opciones ofrecen herramientas y bibliotecas para la comunicación con instrumentos de medición.

Python es un lenguaje de programación de alto nivel, conocido por su simplicidad y legibilidad. Cuenta con una amplia gama de bibliotecas y frameworks que facilitan el desarrollo de aplicaciones científicas y de ingeniería. Además, es de código abierto, gratuito y de uso libre, incluso para fines comerciales, lo que permite acceder a una gran cantidad de recursos sin tener que pagar licencias.

C++ es un lenguaje compilado que destaca por su máximo rendimiento en tiempo de ejecución y control a bajo nivel de los recursos del sistema. Presenta una curva de desarrollo pronunciada, la necesidad de gestionar la memoria manualmente, y la verbosidad de su sintaxis que incrementan drásticamente los tiempos de programación frente a lenguajes de mayor nivel de abstracción. Según el tipo de licencia que se adopte para QT, puede ser gratis o tener un costo asociado.

LabVIEW, es un entorno de desarrollo gráfico utilizado principalmente en aplicaciones de ingeniería y automatización, conocido por su facilidad para crear interfaces de usuario y su capacidad para interactuar con hardware de medición. Sin embargo, es un software propietario, con licencias costosas, que lo convierte en la opción menos accesible.

Las tres herramientas llevan mucho tiempo en el mercado y son ampliamente utilizadas en la industria, por lo que son opciones viables para llevar a cabo este proyecto. Sin embargo, dada la diferencia de costos y la agilidad de desarrollo requerida, Python se presenta como la opción más balanceada, accesible y eficiente para el contexto del laboratorio.

En la Tabla 1 se sintetiza el análisis comparativo que decantó en la adopción conjunto de herramientas: Python 3 para la lógica de negocio, PySide6 para la interfaz gráfica y PyVISA para la comunicación con el hardware.

Criterio de Evaluación	Python	C++	LabVIEW
Tipo de Lenguaje	Interpretado (Alto nivel).	Compilado (Alto nivel/Bajo nivel).	Gráfico (Flujo de datos).
Bibliotecas	Nativas + Externas. Múltiples aplicaciones.	Nativas + Externas. Enfocadas en rendimiento.	Nativas. Especializadas en hardware/instrumentos.
Desarrollo de GUI	QT Designer (<i>Drag & Drop</i>). Hay que codificar las funciones.	QT Designer (<i>Drag & Drop</i>). Hay que codificar las funciones.	Nativo (<i>Drag & Drop</i>). Funciones ya codificadas.
Costos	Gratis. <i>Open Source</i> (GPL/LGPL/BSD).	Depende de la licencia de Qt. <i>Open Source</i> o Comercial.	Licencia costosa.
Mantenimiento	Alto. Código basado en texto.	Alto. Código basado en texto.	Bajo. Archivos binarios (.vi), difícil revisión y actualización de código.

Tabla 6.1: Análisis comparativo de entornos para instrumentación y procesamiento de señales.

6.5 Gestión del entorno de desarrollo y control de versiones

El desarrollo del código fuente se centralizó en el entorno de desarrollo integrado Visual Studio Code (VS Code). Para asegurar la reproducibilidad del entorno de ejecución a través de diferentes estaciones de trabajo y evitar conflictos con los paquetes del sistema operativo, el proyecto se aisló utilizando entornos virtuales nativos (venv). Todas las dependencias (ej. PySide6, numpy, h5py) quedaron registradas explícitamente en un archivo de requerimientos (requirements.txt) con sus versiones exactas. Esta homologación garantiza la utilización de las mismas librerías independientemente de la computadora de desarrollo, certificando así la estabilidad y el comportamiento determinista de los algoritmos matemáticos a lo largo del tiempo.

El control de versiones del código fuente se gestionó mediante Git. Se adoptó un flujo de trabajo basado en la creación de ramas. Esta estrategia permitió:

- Desarrollar características experimentales de forma aislada (ej. implementación de la GUI) en ramas secundarias.
- Integrar (*merge*) el código a la rama principal (*main*) únicamente tras la validación funcional de los módulos frente a las señales de referencia del TDG.

6.6 Gestión de problemas

Para garantizar el funcionamiento continuo y evitar comportamientos indeterminados, ante perturbaciones o errores operativos, la aplicación tiene la siguiente estructura de manejo de problemas:

1. **Errores de comunicación del hardware:** En caso de una desconexión accidental del cable USB, o fallos de alimentación del osciloscopio durante el ensayo, la capa lógica interrumpe la ejecución, emite una notificación al operador sobre la pérdida de comunicación, y reestablece los controles de la interfaz gráfica.
2. **Validación de datos de entrada:** Para impedir que el software realice cálculos numéricos con buffers vacíos o configuraciones incompletas, el presentador realiza validaciones lógicas antes de interactuar con el módulo de almacenamiento o el analizador matemático, previniendo las siguientes fallas:
 - **Ensayo nulo:** Si el operador acciona el botón de «Guardar» o «Exportar» sin haber creado previamente la carpeta del ensayo, se detiene la operación, y se notifica la ausencia del directorio del ensayo.
 - **Ausencia de condiciones ambientales:** Se restringe el ingreso de datos inválidos en estos campos, y se impide el almacenamiento de la onda si no están cargados, advirtiendo al operador que debe completarlos.
 - **Buffer de adquisición vacío:** Si el operador solicita guardar datos sin haber capturado una señal, se interrumpe la escritura de un registro nulo, y genera una alerta de buffer vacío.
3. **Fallos del módulo matemático:** Durante el procesamiento de la señal, pueden presentarse divergencias o inestabilidades numéricas. En ese caso, el motor de cálculo detiene el proceso, y notifica al operador sobre la necesidad de repetir la captura y reajustar la configuración del osciloscopio (si es necesario).

La etapa 1 de acondicionamiento analógico, es propia del montaje del ensayo, por lo que, no se afecta en ningún aspecto. La etapa 2 de digitalización, se modificó para este proyecto respecto al montaje vigente, analizando entre las alternativas de osciloscopio disponibles en el laboratorio, cuál era el más indicado tanto por sus características eléctricas, como por su capacidad de comunicación con la PC. Mientras que, las etapas 3, 4 y 5 son desarrollo íntegro del presente trabajo.

Capítulo 7

Interfaz gráfica de usuario GUI

El diseño de la **Vista** se modela empleando la herramienta visual Qt Designer, que sirve para diseñar interfaces gráficas visualmente (arrastrando y soltando objetos). Luego, una vez que se terminó el diseño de la ventana, se compila a código de Python mediante el compilador de la biblioteca PySide6.

7.0.1 Configuración del osciloscopio mediante la GUI

El operador configura de manera remota el osciloscopio interactuando con la Interfaz Gráfica de Usuario (GUI, *Graphical User Interface*), la cual se comunica con el instrumento, siguiendo un flujo de control basado en la arquitectura MVP.

1. **Captura de eventos (Vista):** La GUI captura las entradas de texto, la selección de menús desplegables o la pulsación de botones, y emite señales de Qt.
2. **Orquestación (Presentador):** El presentador conecta las señales de la GUI con sus correspondientes receptores lógicos (*slots*). Además, valida que las entradas sean correctas, y envía las configuraciones al módulo controlador de *hardware*.
3. **Traducción y control de *hardware* (Modelo):** El controlador de *hardware* traduce los parámetros en comandos SCPI, y los envía al instrumento, que responde en función de la operación realizada.

La interfaz gráfica se divide en los siguientes bloques de visualización:

- **Datos del Ensayo:** Metadatos generales del ensayo (fecha, identificador del elemento bajo ensayo y cliente solicitante).
- **Condiciones Ambientales:** Parámetros ambientales del laboratorio (temperatura, presión atmosférica y humedad).
- **Atenuaciones del Sistema:** Factores de atenuación de la señal analógica (divisor resistivo, resistencia *shunt* y atenuadores coaxiales).
- **Configuración del Osciloscopio:** Vincula y define de los parámetros del instrumento (escalas de amplitud vertical, escala de tiempo horizontal y sistema de

disparo).

- **Ensayo:** Inicia la captura del impulso, además de guardar y exportar los resultados.
- **Resultados del ensayo:** Muestra los resultados del último impulso analizado.
- **Gráfico de onda:** Muestra el historial de ondas almacenadas.

Capítulo 8

Calibración del software

8.1 Validación y Reportes (report.py y Testing)

El proceso de aseguramiento de la calidad del software se ejecuta mediante una suite de pruebas automatizadas que inyectan las formas de onda sintéticas y grabadas definidas en la norma IEC 61083-2 [1]. El objetivo de este procedimiento es cuantificar el sesgo del algoritmo respecto a las magnitudes de referencia metrológica.

8.1.1 Flujo de Validación del Software contra TDG

La validación implementa el protocolo de ensayos de desempeño (performance tests) estipulado en la sección 6.2 de la norma. Se cargan los casos de prueba del generador de datos de prueba (TDG) correspondientes al grupo Full Lightning Impulse (LI-A1 a LI-A12 y LI-M1 a LI-M17) e impulsos cortados en la cola (LIC-M4 y LIC-M5). El sistema procesa los archivos ASCII de dos columnas en segundos y voltios, ejecuta el pipeline algorítmico de filtrado y remoción de componentes continuas, y contrasta los resultados con los límites normativos.

8.1.2 Estimación de Incertidumbre del Software (Anexo B.3.3)

En lugar de adoptar los límites simplificados fijos, el módulo de pruebas calcula las incertidumbres combinadas del software basándose en las desviaciones analíticas reales encontradas frente al TDG, siguiendo de forma estricta las directrices matemáticas del Anexo B de la norma IEC 61083-2 [1].

La primera componente del cálculo representa la incertidumbre estándar tipo B del software (u_{B71}), calculada a partir de la máxima desviación relativa observada en el conjunto de datos analizados (n ondas evaluadas):

$$u_{B71} = \frac{1}{\sqrt{3}} \max_{i=1}^n \left| \frac{x_i - x_{\text{REF},i}}{x_{\text{REF},i}} \right| \quad (8.1)$$

La segunda componente (u_{B72}) considera la incertidumbre expandida intrínseca del vector de referencia del TDG suministrada por los comités internacionales de mantenimiento, calibrada a un nivel de confianza del 95 % con un factor de cobertura $k = 2$:

$$u_{B72} = \frac{1}{2} \max_{i=1}^n U_{x,i} \quad (8.2)$$

La incertidumbre estándar combinada asignada al componente del software (u_{B7}) se obtiene mediante la raíz cuadrada de la suma de los cuadrados de ambas componentes independientes:

$$u_{B7} = \sqrt{u_{B71}^2 + u_{B72}^2} \quad (8.3)$$

8.1.3 Validaciones Paramétricas bajo IEC 61083-2

El software ejecuta la validación de la función de transferencia de los algoritmos mediante el procesamiento de vectores digitales de referencia. El objetivo de este procedimiento DSP es aislar la curva base del impulso de cualquier perturbación de alta frecuencia (*overshoot* o ruidos de conmutación) para construir la curva de tensión de ensayo de manera reproducible. La norma define diferencias máximas aceptables entre los valores verdaderos del generador de datos de prueba (TDG) y los calculados por el algoritmo bajo evaluación. El procesamiento se divide de acuerdo a la topología de la señal bajo análisis:

Tabla 8.1: Tolerancias de aceptación paramétricas según la norma IEC 61083-2.

Tipo de Impulso	Parámetro Físico	Tolerancia Máxima
Completo (LI)	Valor Pico (U_t)	$\pm 0,10 \%$
	Tiempo de Frente (T_1)	$\pm 2,0 \%$
	Tiempo de Cola (T_2)	$\pm 1,0 \%$
	Sobrepasamiento (β)	$\pm 1,0 \%$ (Absoluto)
Cortado (LIC)	Tiempo de Corte (T_c)	$\pm 2,0 \%$

Parte III

RESULTADOS Y CONCLUSIONES

Bibliografía

- [1] International Electrotechnical Commission, *Instruments and software used for measurement in high-voltage impulse tests - Part 2: Requirements for software for tests with impulse voltages and currents*, Geneva, Switzerland, 2013.
- [2] International Electrotechnical Commission, *High-voltage test techniques - Part 1: General definitions and test requirements*, Geneva, Switzerland, 2010.
- [3] Good Will Instrument Co., Ltd., *Digital Storage Oscilloscope GDS-1000A-U Series Programming Manual*, Part No. 82DS-112AUI01, New Taipei City, Taiwan, 2011. dirección: www.gwinstek.com.
- [4] Good Will Instrument Co., Ltd., *GDS-1000A-U Series USB Driver*, Accedido en 2026, GW Instek. dirección: <https://www.gwinstek.com>.